

Домашка 1 · свой сервис от артефакта до кластера

повторить путь семинара для своей модели

Дедлайн: воскресенье 20.09, 23:59. На семинаре 2 поверх этого репозитория строим CI/CD, поэтому без сделанной базовой части будет тяжело.

1 Правила игры

Модель **своя**, не churn из семинара. Подойдёт что угодно: задача из вашей работы или учёбы, рет-проект, любой открытый датасет. Если идей нет, за полчаса найдётся датасет на Kaggle или openml.org: цены на жильё, отток банковских клиентов, задержки рейсов, диабет.

Качество модели не оценивается вообще. Логистическая регрессия или CatBoost из пяти строк это нормально: оценивается инженерная обвязка вокруг. Эталон семинара week1 лежит в архиве, подглядывать можно и нужно, копировать вслепую нельзя: схема фичей, паспорт и тесты у вас будут свои.

2 Базовая часть

Каждый пункт повторяет блок семинара. Порядок тот же, что был в субботу.

2.1 1. Артефакт с паспортом

Обучите модель в ноутбуке или скрипте и сохраните joblib-бандл из двух частей: pipeline (пре-процессинг и модель одним Pipeline) и metadata (версия, список фичей в правильном порядке, порог для классификации). Ноутбук обучения положите в репозиторий.

2.2 2. Проект на uv

Src-layout как на семинаре: src/<пакет>/, pyproject.toml, .python-version. Коммитим и uv.lock. Проверка на чистой машине:

```
uv sync
uv run python -c "import <пакет>"
```

2.3 3. Сервис

FastAPI с теми же четырьмя точками: /health, /ready, /v1/predict, автодоки. Схема запроса описывает **ваши** фичи: границы значений, extra="forbid", опциональные поля там, где в данных бывают пропуски. Модель грузится один раз в lifespan. В ответе: предсказание, версия модели, request_id, latency_ms.

2.4 4. Логи в Postgres

Каждый запрос к /v1/predict пишется строкой в таблицу: request_id, время, версия, фичи как jsonb, предсказание, latency_ms и **код ответа**: это поле новое относительно семинара, добавьте его сами. Без DATABASE_URL сервис работает, просто не логирует.

2.5 5. Тесты

Минимум шесть, три обязательных сюжета: контракт (мусор и лишние поля дают 422), smoke (ответ в допустимом диапазоне, правильные типы), детерминизм (один вход дважды даёт один ответ). uv run pytest зелёный.

2.6 6. Docker и compose

Dockerfile на uv: слой зависимостей до слоя кода, --frozen --no-dev. compose.yaml с сервисом и Postgres, у базы healthcheck, у api depends_on с условием. После docker compose up -d --build predict логируется, SELECT из таблицы показывает строки.

2.7 7. Kubernetes

Манифесты k8s/deployment.yaml и k8s/service.yaml: две реплики, три пробы на ваши /health и /ready, requests и limits. В kind: загрузить образ, применить манифесты, дождаться rollout, через port-forward получить ответ модели.

2.8 8. README

Ровно три команды проверки: тесты, compose, kind. Проверяющий делает git clone и выполняет их сверху вниз, ничего не редактируя.

К сдаче: базовая часть

Ссылка на репозиторий. В README, кроме трёх команд, четыре доказательства **скринами**: терминал с выводом pytest, SELECT из таблицы логов, kubectl get pods с 2/2 Running вместе с ответом /v1/predict через port-forward, и k9s с вашими подами. Требования к отчёту целиком: раздел 4.

Куда смотреть, если застряли

Маршрут семинара walkthrough01.pdf: там каждый шаг с командами и аптечка типовых поломок. Шпаргалка команд seminar01_students.pdf. Тег #stuck в чате работает и между семинарами.

3 Дополнительная часть

Три задачи, каждая независима и тянет на отдельную звёздочку. Делать можно любую, можно все.

3.1 Звёздочка 1 · Нагрузочное тестирование

На семинаре мы это пропустили, поэтому разберём подробнее. Locust гоняет виртуальных пользователей по вашему API и считает статистику латентности.

Как делать:

```
| uv add --dev locust
```

Напишите locustfile.py под свою схему: класс пользователя, задача с POST на /v1/predict и валидным телом, вес побольше, и задача с GET /health, вес поменьше. Пример структуры есть в эталоне.

Поднимите сервис через compose и сделайте три прогона с ростом нагрузки:

```
| uv run locust -f locustfile.py --headless -u 10 -r 5 -t 60s --csv run10 -H http://127.0.0.1:8000
| uv run locust -f locustfile.py --headless -u 50 -r 10 -t 60s --csv run50 -H http://127.0.0.1:8000
| uv run locust -f locustfile.py --headless -u 100 -r 20 -t 60s --csv run100 -H http://127.0.0.1:8000
```

Флаг --csv сохранит таблицы отчёта. Если убрать --headless, на localhost:8089 откроется веб-интерфейс с живыми графиками: для первого знакомства он нагляднее.

Что положить в отчёт: таблица по трём прогонам со столбцами RPS, median, p95, max, доля ошибок, и три-четыре предложения выводов. Интересные вопросы: на каком числе пользо-

вателей p95 оторвался от медианы; растёт ли RPS при переходе от 50 к 100 пользователям или сервис уже упёрся; появились ли ошибки и какие.

Подсказка

Нагрузку гоните на compose-версию, не через `kubectl port-forward`: туннель сам станет узким местом, и мерить вы будете его, а не сервис.

3.2 Звёздочка 2 · Батч-эндпоинт с замером

Добавьте `/v1/predict/batch`: схема с `rows: list[Features]` и границами от 1 до 1000, из всех строк собирается один `DataFrame`, модель вызывается один раз. Код показывали в бонусе, он есть на слайде 18 преподавательской презентации, но у вас схема своя.

Замерьте: одна строка против 500 строк в одном запросе. Проще всего посмотреть `latency_ms` из ответа сервиса, для честности возьмите медиану из десятка повторов. К сдаче: оба числа и два предложения: во сколько раз 500 строк дороже одной и почему настолько мало.

3.3 Звёздочка 3 · Выкат новой версии и откат

Соберите образ 1.1 с любым видимым отличием: подойдёт батч-эндпоинт из звёздочки 2 или новое поле в `/health`. Дальше путь из бонуса:

```
docker build -t <образ>:1.1 .
kind load docker-image <образ>:1.1 --name <кластер>
kubectl set image deploy/<деплой> <контейнер>=<образ>:1.1
kubectl rollout status deploy/<деплой>
kubectl rollout undo deploy/<деплой>
kubectl rollout history deploy/<деплой>
```

Пока идёт выкат, держите рядом `kubectl get pods -w` или `k9s` и смотрите, как поды меняются по одному. К сдаче: вывод `rollout history` и два-три предложения: что происходило с подами во время выката, что вернул откат и почему сервис не молчал ни секунды.

4 Отчёт

Отчёт обязателен и живёт в репозитории: раздел в `README` или отдельный `REPORT.md`. Что в нём должно быть:

- **Скрины терминала** на каждый чекпоинт базовой части: `pytest`, `SELECT` из логов, `get pods`, ответ `predict` через `port-forward`. Именно скрины экрана, не перепечатанный текст.
- **Скрин k9s** с вашими подами: обязательно. Подойдёт вид `:pods` или `:xray deploy`.
- **Журнал проблем**: что не получилось с первого раза, текст ошибки, как починили. Если что-то не завелось совсем, так и пишите, но с диагностикой: что сказали `describe` и `logs`, что пробовали. Честное «не завелось, вот что я выяснил» засчитывается, молчаливо пропущенный пункт нет.
- Для сделанных звёздочек: числа и выводы, как описано в задачах.

5 Сдача

Одна ссылка на репозиторий в личку или в форму, как договорились в чате. Базовая часть и отчёт обязательны, звёздочки по желанию, каждая отмечается отдельно. Главный критерий один: `git clone` и три команды из `README` работают у проверяющего с первого раза. После дедлайна репозиторий можно дорабатывать, но проверяется состояние на момент сдачи.

Дедлайн: воскресенье 20.09, 23:59.